



 | SYNTASSO  
KRATIX  
ENTERPRISE



## Background

**Effective internal platforms help you go faster, be safer, and increase efficiency as you scale.**

To create an effective platform, you must incorporate three essential principles:

1. Self-service and on-demand access
2. Consistently applied business processes
3. Fleet-manage services at scale

Syntasso Kratix Enterprise (SKE) does this and more by providing the Promise API. Use this card deck to design your own services that comply to all the platform principles.



# Interface

## Interfaces explained

**Interfaces enable users to create, update, delete, maintain, and review platform capabilities.**

There's no single right way to access a platform—some tasks require scripting, while others rely on visualisation, each needing different interfaces.

The key is ensuring your platform supports all interface types with minimal maintenance.



# Interface

## API *(default)*

All modern platforms today start with a powerful API and layer in additional interfaces on top. For example, cloud providers start with the API and then extend to a CLI and GUI for different use cases.

Therefore, this interface is labeled as "default" and should always be included in the interfaces section.



# Interface

## Portal

Often browser-based (GUI) interfaces that show a catalog of platform services and resources.

Portals can be custom built, self-hosted projects (e.g. the popular CNCF project Backstage) or SaaS products (e.g. Port, Cortex, and Roadie).

Portal configuration and data is directly dependent on the platform and therefore should be managed by the platform itself to reduce overhead.



## Ticketing System

Ticketing systems are commonly associated with long wait times and manual processes but are also often core to an organisation's user experience and tracking mechanisms.

Platforms can be exposed via the existing tools such as Jira and ServiceNow to maintain experience and tracking but replace manual processes with on-demand offerings.



# Interface

## Command Line Interface (CLI)

A CLI enables terminal-based platform engagement which is often a preference for engineers and even a requirement for some tasks such as CI/CD scripts which do not have browser access.

CLI configuration and data is directly dependent on the platform and therefore should be managed by the platform itself to reduce overhead.



# Interface

## Integrated Developer Env (IDE)

Developers are often working with AI and code in an IDE, whether it be more graphical (e.g. JetBrains or VSCode) or in the terminal (e.g. VIM or Neo).

Platforms can provide MCP servers and plugins to any IDE to provide faster feedback and easier access to tools and developer environments.



# Destination

## Destinations explained

**Destinations are the locations declarative code can be scheduled to.**

A platform has many destinations. Each one refers to a target location that can have declarative code applied via GitOps or other deployment process.

There are countless types of destinations as they can be scoped by type of technology, environment, or ownership model.



# Destination

## Kubernetes cluster

There can be as many destinations as you have clusters. Each one configured to the right level of security and scale for its use case.

Each destination can refer to environments (dev/prod), type (GPU, PCI compliant), or any other domain model you divide clusters by within your organisation.



# Destination

## Cloud Provider

You may require multiple cloud destinations to cover different providers, accounts, even regions.

Each destination represents a unique domain and can be configured using proprietary tools (e.g. CloudFormation or Bicep) or general Infrastructure as Code tools such as Crossplane or Terraform



# Destination

## Software as a Service (SaaS)

This can represent any number of vendors where the software you depend on is run by the vendor and accessed over the internet.

You can use this destination to manage what vendor features are available as well as to provide access or features for a specific tenant.



# Destination

## On-premise / hardware

Not all resources are in the cloud.

If you need to manage on-premise infrastructure you can use something as simple as a cronjob to refer to the declared state in a Destination to maintain and update the configuration of anything from a network device to a server to a mainframe.



# Input

## Inputs explained

**Inputs are the pieces of information that the platform users can configure.**

Input fields can be in many different formats, such as enums, strings, numbers, and booleans.

Keep in mind, inputs are how users interact with the platform, so they should be viewed as a collaborative effort.



# Input

## Name

This is both the most common, and often the least necessary input, so think twice before using. Do you require a name from the user? Or is this something you can generate?

Name inputs are most important when the name is frequently referenced off-platform.



# Input

## Size

How big a resource is can affect its cost and efficiency. Users may be able to indicate their requirements in bucket sizing (e.g. small, medium or large) or need to provide specific bytes.

Keep in mind that if size is provided, and it is either too big or too small, it will be the user who needs to update their request to match the actual need.



## Physical location

Most of the time a user doesn't care *where* a resource is deployed, just that it is available through networking.

In some cases though a user must meet data residency requirements and/or latency requirements which demand a specific physical location for their resources.



## Container image

If a user is asking the platform to deploy software, it likely needs to provide the software to deploy. One option is to provide a previously built image via its registry or other indicators.



## Security level

Not all resources require the same level of security. For example, some resources may never touch customer data, others may contain not only customer information, but data that can be used for financial or other personal attacks.

If the platform can not derive the required security level a user can provide this information.



## Dependencies

A resource may have one or more optional dependencies. For example, a software application can opt into feature flagging services or a database.

Users can provide a list of dependencies that the platform can access existing dependencies of create and then manage new instances in relation to the requested resource.



## Upgrade rules

The power of a platform is that it can manage upgrades as an entire fleet. But just because the whole platform can be updated at the same time doesn't mean it should be.

Some users may want to opt into a slower or later stage upgrade schedule while others may want the cutting edge releases.



# Input

## Repository

If a user is asking the platform to deploy software, it likely needs to provide the software to deploy. One option is to provide a repository where the software lives and allow the platform to derive the deployment from there.



## Ownership

Ownership can be used for a lot of different things including tagging and access permissions.

In the event that the platform can not determine ownership from the requesting user, it may need to be explicitly told in order to adhere to business processes.



# Input

## Version

Ownership over the version of a resource implicitly determines ownership of upgrades and maintenance.

For example, if a platform exposes version as an input, the users are then required to engage with any upgrades of the resource. But if the platform only exposes the major version it would implicitly be owning the management of both minor and patch version updates.



# Input

## Configuration

Configuration is a generic reference to any fields which alter the behaviour of a resource within the defined boundaries.

If you choose to use this input, think about adding notes about what specific configuration you would expose.



# Input

## Environment

It is common to have a number of distinct setups or configurations where software is built, tested, and deployed.

While environments can require the same infrastructure and tooling building blocks, they often implement them in different ways. Provided this input, the platform can determine these configurations for the user.



# Output

## Outputs explained

**Outputs are the information that the platform reports back to users both on creation and over time.**

It is common to include some data that is designed for humans to read and other data that is designed for machines to parse.

These outputs should be sufficient for users to both use and debug the resources.



## Secrets

When a resource requires a secret for access (e.g. a password or a database connection string), the users will need to know what that secret is.

API responses are not secured transit, however you can return back a location where the secret is available. E.g. a HashiCorp Vault or cloud secret manager URL.



## Observability details

Platforms create a shared responsibility between the users and the platform providers. Users must track their resources and make sure they have requested the right configuration (e.g. size or security level).

To manage these requirements, the platform may provide necessary observability data and tooling access.



## Access information

Resources are often physically depended on by a user or other system and outputs can provide the details for downstream dependencies.

Users may need to know a URL or endpoint to run manual tests. Or another resource may need to know an IP address or other identifying information to connect and create a higher level system before it is useful to the user.



## Health and status

"Are we there yet?" isn't just a question for long drives. One of the most common questions of a platform is if a resource is available to be used for the first time and over its lifespan.

Health updates can alert users to over or under-provisioning, security alerts, and other available operational improvements including debugging support when an issue occurs.



## Deprecations

Services need support over time. Sometimes updates require user action, other times they can be done completely by the platform but a user may want to be informed.

Outputs can alert users to known deprecated versions and be clear about when and how an upgrade will occur.



# Process rules

## Process rules explained

**Process rules are business tasks executed alongside resource provisioning.**

Processes rules can be thought of as part of a todo list if someone had to respond manually to a request via a request ticket or could be codified in a delivery pipeline.

These can be used in either the dependency setup or per resource configuration (or both!).



# Process rules

## Security scan

Resources often need static scanning before deployment and regular dynamic scanning during use.

This scan can be used for either, or both, of these scans as it can be run on lifecycle events as well as on a regular schedule.

The platform can also execute different actions based on the results of a scan.



# Process rules

## Status check

Sometimes in the process of a resource creation, update, or delete the platform may need to know the status of either that resource or others.

For example, the platform may have a rule to not do deployments during an open priority zero incident, or to only run one upgrade at a time.



# Process rules

## Fetch available options

Options may be dynamic and need to be validated on provisioning.

For example, a platform load balances ephemeral test environments and the platform can fetch current test environment state to determine where the new application should be deployed. Or a platform may need to use an external LDAP service to manage authorisation.



# Process rules

## Notification

Chatbot, SMS, email or other notifications can be sent to both the user who requested a resource as well as other interested parties.

Notifications can be used to inform and alert users. If the resource requires auditing, look to use the *reporting* process action.



# Process rules

## Budget check

Organisations may exercise FinOps where they track user, team, or organisational costs over time. The platform can check for compliance with FinOps requirements and choose to either allow, disallow, or alert for approval on resources based on their compliance.



# Process rules

## Status update

It is common to update status at the end of provisioning, but sometimes the platform may want to also update status throughout the process to indicate progress.

Status can be informational or indicate required action.



# Process rules

## Manual approval

While we may all aspire to complete automation, there are many compliance and historic reasons why manual approval is, and should, still be applied.

This process can pause and wait until an authorised person approves the request.



# Process rules

## Resource tagging

Tags or labels on resources help to identify their ownership and more.

These labels not only help operations in supporting the resources, but also finance, security, and other departments when running audits on the organisation wide resources.



# Process rules

## Scheduling rules

When using declarative provisioning, the resources need to be scheduled to the right Destinations.

By default, a resource will be scheduled to one of the available Destinations, but in some cases the platform will apply more specific rules to control the specific Destination it is scheduled to.



# Process rules

## Deprecation management

Platforms need to manage the identification of out of date dependencies and then follow a process to determine urgency, notify affected parties, and manage a safe upgrade.

The platform will need to adhere to compliance requirements for all new and existing resources on the platform.



# Process rules

## Reporting

If you need to track user requests due to compliance regulations or other requirements, you can choose to send audit events on requests and changes within the platform. In addition, the platform can self-report on a regular basis about the state of its resources.

If you are just looking for user awareness, check *notifications* for another option.



# Provisioning action

## Provisioning actions explained

**Provisioning actions produce a tangible, useable platform service.**

Provisioning can be imperative or declarative. Imperative executes step-by-step instructions. Declarative defines desired state. Destinations are required for declarative provisioning.

These can be used in either the dependency setup or per resource configuration (or both!).



# Provisioning action

## Infrastructure

Infrastructure is the cloud or physical hardware that can run compute, data storage, networking, and other resources.

Infrastructure can be provided as a complete unit (think a whole apple pie), or as a part of a shared unit (think a slice of pie). In either case, you are allocating the defined infrastructure. Examples could be servers, databases, or Kubernetes Clusters.



# Provisioning action

## Software as a Service (SaaS)

Manage a SaaS product configuration. For example, adding user accounts, creating resources, or updating settings.



# Provisioning action

## Kubernetes cluster

Resources can be deployed to Kubernetes and may require creating the Kubernetes cluster to be used for additional provisioning.



# Provisioning action

## Credentials

Create or access credentials to be used to create and then access the requested resource.

Credentials can be used by one or more users for one or more use cases. Therefore, the credentials may need to be accessed from a number of locations or be referenced by a number of requests.



# Provisioning action

## Cloud account

Resources can be deployed to a cloud provider and may require creating the account to be used for additional provisioning.



# Provisioning action

## Kubernetes namespace

Resources can be deployed to Kubernetes and may require creating the Kubernetes namespace to be used for additional provisioning.



# Provisioning action

## Configuration

Configuration refers to the unique settings for a resource.

A resource may have global configuration or per user specific configuration. Depending on the use case, configuration may need to be combined with other configuration for other requests.



# Provisioning action

## Git repository

If the platform offers bootstrapping of a new service, it is common to also provide an initial example application and deployment.

This action can be done imperatively, simply creating the repository and handing it over to the user, or it can be done declaratively where the platform maintains awareness and even control over certain configurations which provides ongoing support.



# Provisioning action

## Identity & Access

IAM (aka Authz) is a set of rules and permissions that ensure the right people have access to the right resources, and only those resources.

Some rules may apply across a number of users and others may be more specific. For example, requiring 2-factor authentication for all users but accessing a specific instance may be limited to the requesting user.



# Provisioning action

## Existing resources

Some resources share a lifecycle with a number of dependencies. For example, an application may always be provisioned and decommissioned along with its database.

In this case the platform may choose to accept a single request for an application and manage database only in relation to that application.